

PulseTrack AI - Phase 7

Simple Step-by-Step Flow and Working

This version intentionally uses plain text only. No diagrams. It explains what is built, what happens internally, and how the three features connect.

1. Phase 7 in simple words

- **API keys:** let scripts and servers use PulseTrack without human login.
- **Webhooks:** let PulseTrack send outage and recovery events to another system.
- **Audit logs:** let the Owner see who did what and when.

Easy memory rule: API key = program talks to PulseTrack. Webhook = PulseTrack talks to program. Audit log = record of important actions.

2. API Key - complete flow

2.1 Create key

1. Owner opens **Settings -> API Keys**.
2. Owner clicks **Create key** and enters a name such as **CI deploy**.
3. Backend generates a strong random secret, for example `pk_live_aBc9 . . .`
4. The full key is shown once in a modal. The user must copy it now.
5. Database stores only the key prefix for lookup and a SHA-256 fingerprint of the full key. The raw key is not stored.

2.2 Where the key is used

- The user puts the key in a secure place such as a GitHub Actions secret or server environment variable.
- A script sends it in the HTTP Authorization header: `Authorization: Bearer pk_live_ . . .`
- The key is for machine-to-machine access. It is not the browser login token.

2.3 What happens when a request arrives

1. FastAPI reads the Authorization header.
2. The backend extracts the API key prefix.
3. Database lookup finds the matching API-key row.
4. The backend hashes the provided full key with SHA-256.
5. The new hash is compared safely with the stored fingerprint.
6. If valid, the backend finds the key creator and their team/role.
7. The creator must still be active.
8. Normal permission checks run. The key inherits the creator role in v1.
9. The requested route executes.
10. The important action is written to `audit_logs`.

2.4 Success and failure

Valid key: request continues and may return 200/201/etc. **Missing, invalid, or revoked key:** return 401. **Valid key but insufficient role:** return 403.

2.5 Revoke key

- Owner clicks Delete/Revoke for the key.
- The API-key row is removed or disabled according to the implementation.
- The next request with that key fails authentication.
- Because the raw key is not stored, the deleted key cannot be recovered from the database.

2.6 JWT versus API key

Use case	JWT	API key
Human browser login	Yes	No
CI / cron / backend service	Not recommended	Yes
Typical lifetime	Short-lived	Long-lived until revoked
Stored in browser localStorage	Web auth design dependent; not the API key pattern here	

3. Outbound Webhooks - complete flow

3.1 Add a webhook

1. Owner opens **Settings -> Webhooks**.
2. Owner pastes a URL created by the external system they want PulseTrack to notify.
3. Example: `https://myserver.com/pulsetrack-webhook`.
4. For testing, the Owner can use a temporary URL from **webhook.site**.
5. PulseTrack saves the URL, an active flag, and a signing secret.

3.2 Why a worker is used

- A webhook receiver may be slow, unavailable, or return an error.
- The monitoring checker should not wait for that external system.
- So the checker creates a background delivery job.
- ARQ/Redis stores the job and a worker performs the HTTP request.

3.3 Incident opened flow

1. Checker detects the monitored service is DOWN according to incident rules.
2. PulseTrack opens or updates the incident.
3. A webhook delivery job is queued with the incident id and event name **incident.opened**.
4. Worker loads the incident, monitor, team, and active webhook URLs.
5. Worker builds the JSON payload with event, team, monitor, incident, and timestamp.
6. Worker creates an HMAC-SHA256 signature using the webhook secret and timestamp + body.
7. Worker sends an HTTP POST to the saved webhook URL.
8. Request includes X-Signature and X-Timestamp headers.
9. 2xx response means successful delivery; failure/timeout is retried.
10. Delivery result is written to `audit_logs`.

3.4 Why HMAC signing is used

- PulseTrack and the receiver share a secret.
- PulseTrack uses that secret to create a signature from the timestamp and exact request body.
- The receiver calculates the signature again using its copy of the secret.
- Matching signatures prove the request was created with the shared secret and the signed body was not changed.
- The timestamp is checked for freshness, for example within a 5-minute window, to reduce replay attacks.

3.5 Retry logic

- If the destination returns 2xx, mark the delivery as successful.
- If it returns 5xx, times out, or cannot connect, retry.
- Use increasing delays such as 30 seconds, 2 minutes, and 10 minutes.
- Keep the retry work in the background worker so the checker remains fast.

3.6 Recovery flow

1. Later, the checker sees the service working again.
2. PulseTrack resolves the incident.

3. A webhook job is queued with event **incident.resolved**.
4. The same signing and delivery process is used.
5. External systems can use this event to close their own ticket/incident.

3.7 Test webhook

- Owner clicks **Test** in Settings.
- PulseTrack creates a fake test payload instead of waiting for a real outage.
- It signs the payload and sends it through the same delivery path.
- webhook.site or the customer endpoint shows the request immediately.

4. Audit Trail - complete flow

4.1 What is already happening

- Important write actions already call the audit repository.
- Each record can include action, team, user, target, details, IP, and time.
- Phase 7 exposes these records to the Owner through a reader API and Settings UI.

4.2 Audit reader flow

1. Owner opens **Settings -> Audit Logs**.
2. React calls GET /audit - logs with optional filters and pagination.
3. FastAPI verifies the requester is an Owner.
4. If not Owner, return 403.
5. Database query returns matching audit rows.
6. React displays the results in a table.

4.3 Filters and pagination

- user= filters by actor.
- action= filters by event, such as monitor.deleted or webhook.delivered.
- page= and limit= prevent returning huge amounts of data at once.

4.4 Immutability rule

Audit rows are intended to be insert-only. There is no normal update/delete endpoint for audit history. The goal is to preserve an honest record of past actions.

5. Complete Phase 7 story from start to finish

1. Owner creates an API key in Settings and copies it once.
2. Owner creates a webhook in Settings by pasting a URL from their own server, automation service, or webhook.site.
3. Owner tests the webhook and confirms that signed JSON arrives.
4. A script or CI system uses the API key to call PulseTrack.
5. PulseTrack authenticates the key, finds the creator, checks their role, performs the request, and writes an audit record.
6. A monitored site later goes DOWN.
7. The checker opens an incident and queues a webhook delivery job.
8. Worker signs and sends incident.opened to each active webhook.
9. Failures are retried with backoff; results are logged.
10. The site recovers; incident.resolved is queued and delivered.
11. Owner opens Audit Logs and can see key actions, monitor actions, webhook actions, and delivery results.

6. Build order

- **7.1 Keys backend:** migration, CRUD, dual auth, permissions, audit.
- **7.2 Webhooks backend:** migration, CRUD, test endpoint, signing, worker job, retries, checker hook.
- **7.3 Audit reader:** Owner-only GET endpoint, filters, pagination.
- **7.4 Frontend:** Settings sections for Keys, Webhooks, and Audit Logs.
- **7.5 Tests and docs:** authentication, permissions, signing, retry, audit, curl examples.

7. Important security concepts

- **API key is a secret:** show it once; do not store the raw value.
- **Hash instead of storing secret:** keep only a one-way fingerprint.
- **Prefix for lookup:** find the right row without revealing the secret.
- **Role inheritance:** the key acts with the creator permissions in v1.
- **HMAC:** proves webhook authenticity and protects the signed body from modification.
- **Timestamp freshness:** reduces replay attacks.
- **Background job:** webhook delivery cannot block the monitoring checker.
- **Audit immutability:** preserve the history of security-sensitive actions.

8. Manual testing checklist

Test	Expected result
Create API key	Key shown once; prefix/hash stored
Use valid key with curl	Protected endpoint returns data
Use wrong key	401 Unauthorized
Revoke key and retry	401 Unauthorized
Use valid key with wrong role	403 Forbidden where applicable
Create webhook	URL saved; secret created
Click Test	Webhook receiver gets signed JSON
Force incident.opened	Webhook delivered
Return 500 from webhook	Worker retries
Resolve incident	incident.resolved delivered
Open Audit Logs as Owner	Rows visible
Open Audit Logs as Member	403 Forbidden

One-line summary: Phase 7 gives PulseTrack machine access through API keys, automatic outgoing notifications through signed webhooks, and an Owner-facing history of important actions through audit logs.